

1. Introducción

Nuestro proyecto consiste en un robot equilibrista de dos ruedas, es decir, en un robot que mediante el uso de dos ruedas debe mantener el equilibrio.

(INSERTAR IMAGEN 3D Y FOTO DEL ROBOT)

El modelo matemático de este sistema es el del péndulo invertido de dos ruedas (ver fig 1.a). Este sistema es inestable, por lo que el principal desafío es implementar el sistema de control que lo mantenga equilibrado. Un ejemplo comercial de este tipo de robots es el Segway (fig 1.b)

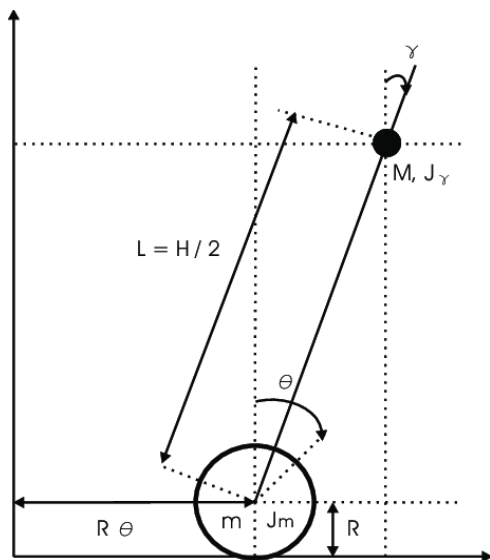


Fig 1.a: Diagrama del péndulo invertido



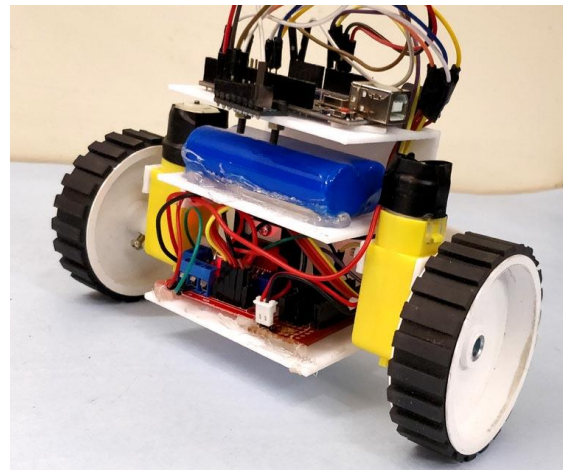
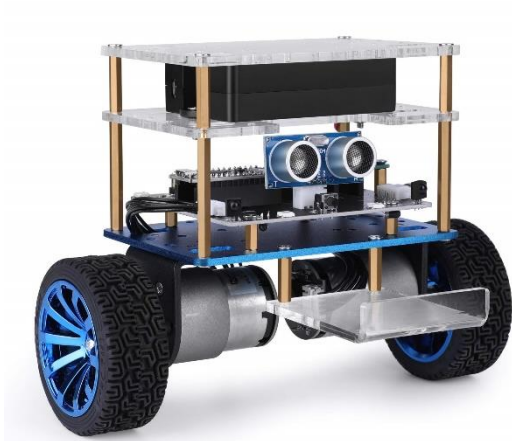
Fig: 1.b: Segway

Las dos variables medidas son, el ángulo de inclinación del robot y su velocidad lineal.

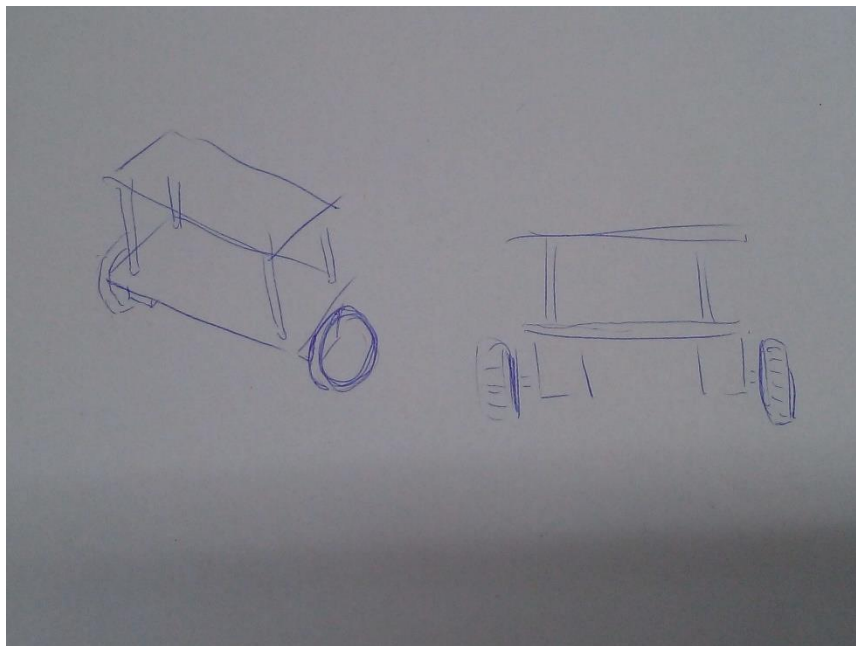
2. Desarrollo de la práctica

Partimos de la idea de hacer un robot que utilizando únicamente dos ruedas pudiera mantenerse equilibrado.

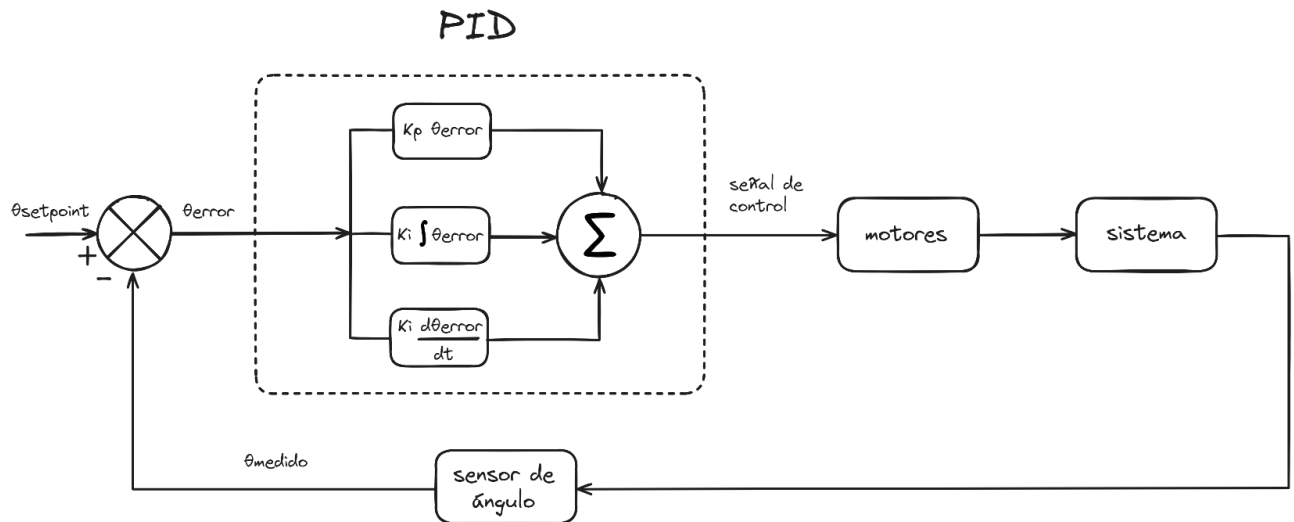
Lo primero que hicimos fue ver otros proyectos del mismo tipo en internet, como estos por ejemplo:



Y de ahí hicimos nuestro primer boceto:



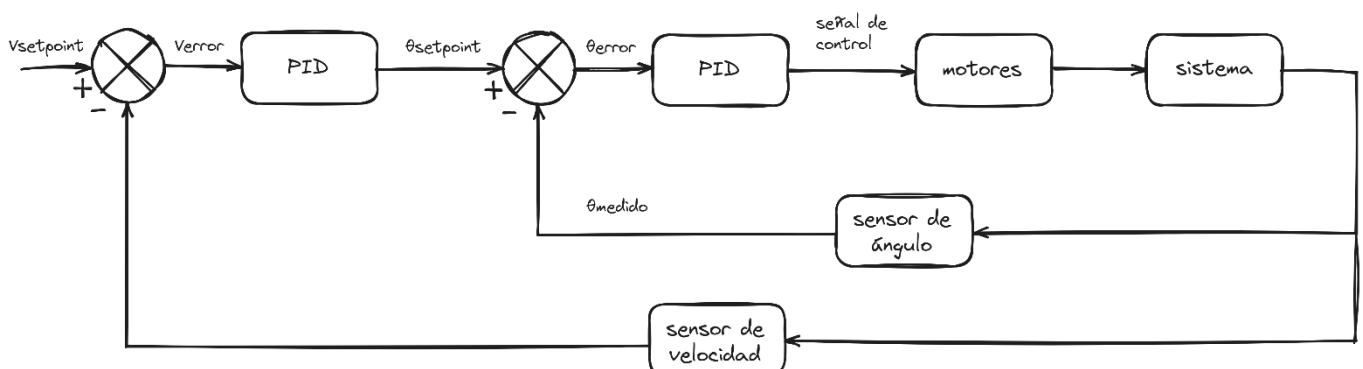
Nuestro robot puede pensarse como un péndulo invertido, siendo un sistema inestable que requiere de un algoritmo de control para mantenerse equilibrado. Muchos proyectos implementaban un control PID para controlar el ángulo de inclinación que medían a partir de un giróscopo/acelerómetro, enviando la señal de control a los motores:



En este caso se establece el setpoint en 0 para que el robot trate de mantenerse vertical (teniendo en cuenta que θ representa el ángulo entre el equilibrista y la vertical..

El problema de este sistema es que, aunque el error se mantenga dentro de cierto rango pequeño, esto no nos garantiza que el robot se quede quieto. Es decir, puede equilibrarse pero se desliza de un lado a otro.

Una forma de resolver esto es añadiendo otro sistema de control en cascada, esta vez, teniendo en cuenta la velocidad lineal del robot, como puede verse en la siguiente imagen:



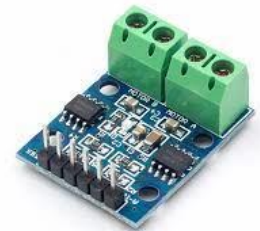
Este algoritmo es usado en los robots más profesionales, como los Segway por ejemplo, y es el que implementaremos nosotros.

2.1. Selección de componentes

Evidentemente necesitamos dos motores. En nuestro caso ya poseíamos dos motores DC de 12V, por lo que terminamos usando esos. En taller medimos una corriente máxima de 500mA por motor (poniendo resistencia en el eje), pero queríamos darle por lo menos 1A, ya que al arrancar siempre hay unos pequeños picos de corriente.



Cada motor debe poder girar en ambos sentidos, por lo que requerimos de un driver para motores. Escogimos el L9110s, ya que aguanta hasta 12V, 2A por motor. (Ver hoja de datos)



Para implementar nuestro sistema de control es conveniente usar un microcontrolador, ya que nos facilita los cálculos y nos permite establecer una comunicación serial. El que elegimos es el ESP32, que claramente puede efectuar los cálculos matemáticos necesarios, además de tener comunicación Bluetooth integrada, que tenemos pensado usar para comunicarnos desde la computadora, a fin de graficar ciertas variables y calibrar las constantes PID.



Para la alimentación utilizamos baterías 18650 a recomendación de los profesores. Como necesitábamos 12V utilizamos 4 baterías en serie, para llegar a 14,8-16,8V, y luego lo regulamos con un step-down. Elegimos un step-down por su eficiencia, ya que necesitábamos 2A para controlar ambos motores, y nuestras baterías eran de 2200mAh, por lo que no podíamos perder potencia innecesariamente usando un regulador lineal. El módulo que

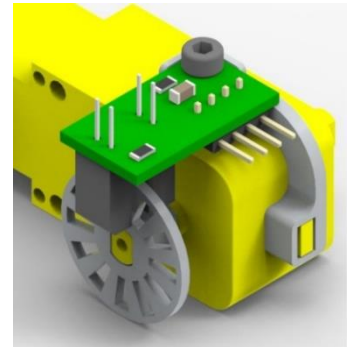


compramos utiliza el integrado XL4005. .

Para la estimación del ángulo, el módulo usado en todos los proyectos que vimos era el giróscopo/acelerómetro MPU6050.

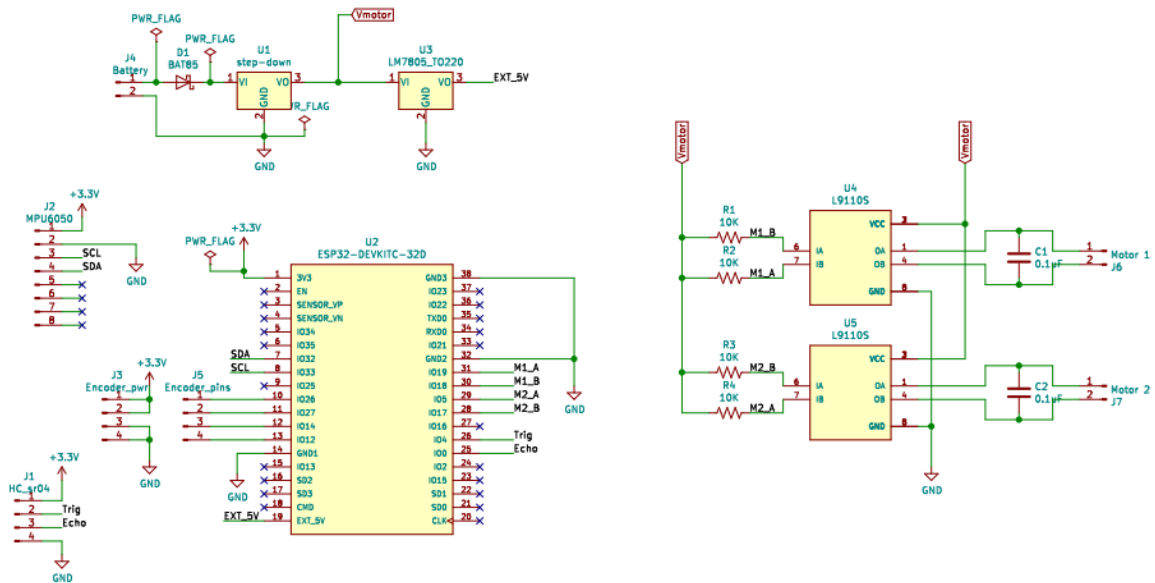


Luego, para medir la velocidad lineal, podíamos medir la velocidad de los motores que es proporcional. Para ello utilizaríamos encoders. Nosotros decidimos diseñar un encoder óptico de dos canales a partir del diseño de un tp de otro curso de 5to año.



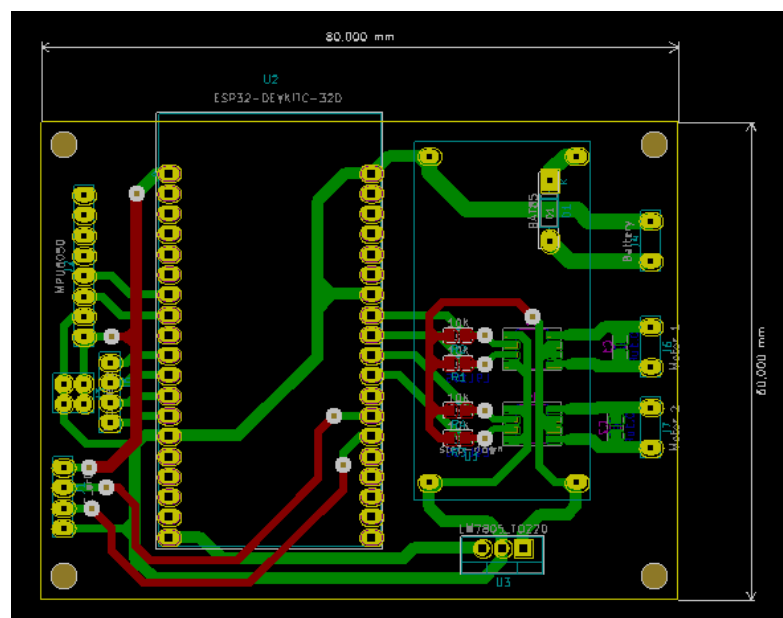
2.2. Diseño y fabricación del PCB

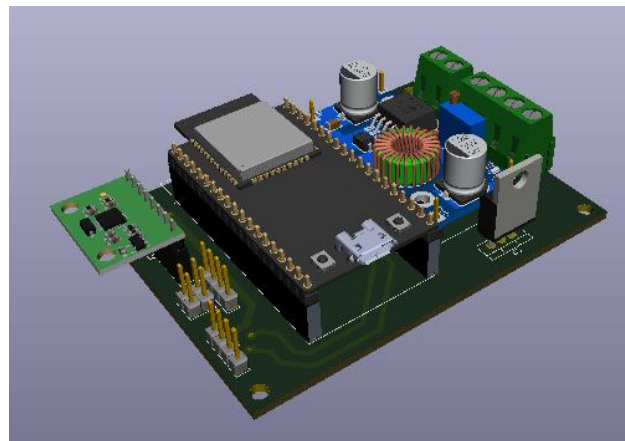
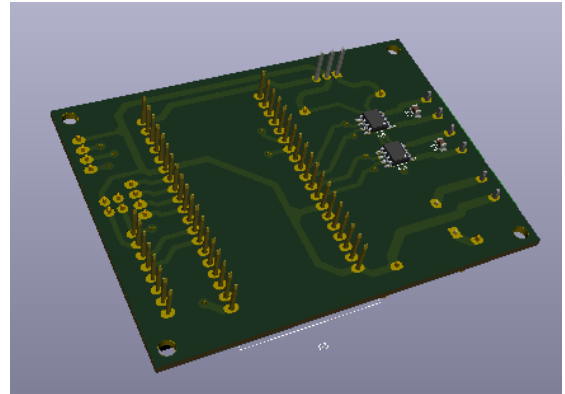
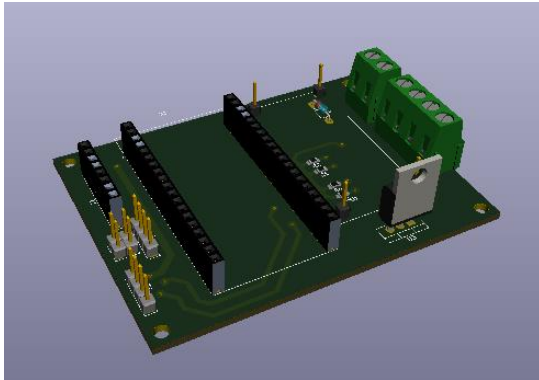
Nuestro PCB debe incluir entonces, borneras para conectar los motores y las baterías, el step-down para alimentar los motores, el puente H, un regulador de 5V para el ESP32, el ESP32 y salidas para conectar los sensores. El esquemático es el siguiente:



Se puede ver que añadimos un diodo Schottky BAT85 para proteger de tensiones inversas y un conector para un sensor de proximidad (HC-sr04).

Y el PCB:

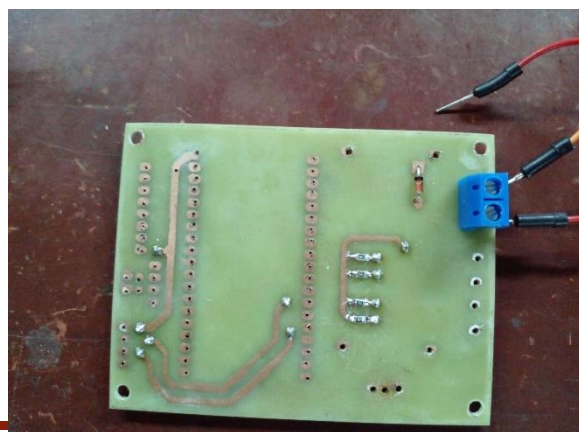
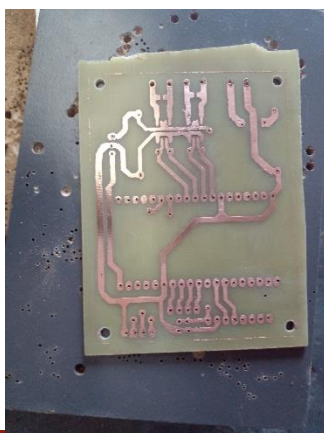




Lo que decidimos es desoldar los componentes del driver para motores y soldarlos a la placa, pero dejar el módulo step-down intacto, soldado encima de la placa, ya que al poseer vías térmicas es más seguro así. Los conectores para los distintos sensores son tiras de pines. Los modelos 3D son o de la librería de Kicad o de Grabcad.

Para hacer este diseño vimos como se conectan el MPU6050 y los encoders. En nuestro caso vamos a usar dos encoders de dos canales cada uno, por lo que se necesitan cuatro pines del ESP32.

Finalmente se planchó, procesó y agujereó el PCB:



Mientras soldabamos, íbamos midiendo las tensiones para comprobar que todo funcione bien.

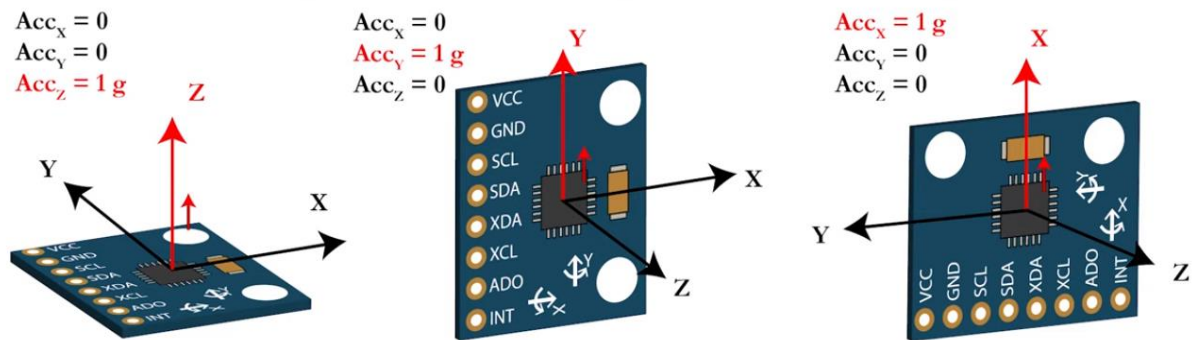


2.3. Medición del ángulo de inclinación

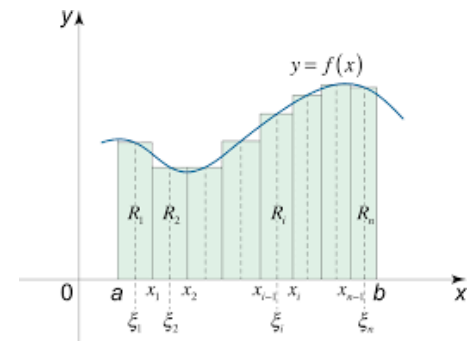
El primer sensor que debemos probar es el MPU. Este sensor se comunica por I2C. Nosotros utilizamos la librería de Arduino Wire.h para implementar esta comunicación. para medir el ángulo de inclinación de nuestro robot.

Lo que nos da el MPU es la aceleración (acelerómetro) y la velocidad angular (giróscopo) para todos los ejes. Con estos dos datos hay dos formas de estimar el ángulo: Por trigonometría o por integración

La primera consiste en despreciar toda fuerza externa a la gravitacional, asumiendo que el vector de aceleración que se mide es el de la gravedad. A partir de este vector, podemos por trigonometría obtener los ángulos de inclinación



Por otro lado, a partir de las medidas del giróscopo, podemos integrar la velocidad angular y también obtener una estimación del ángulo



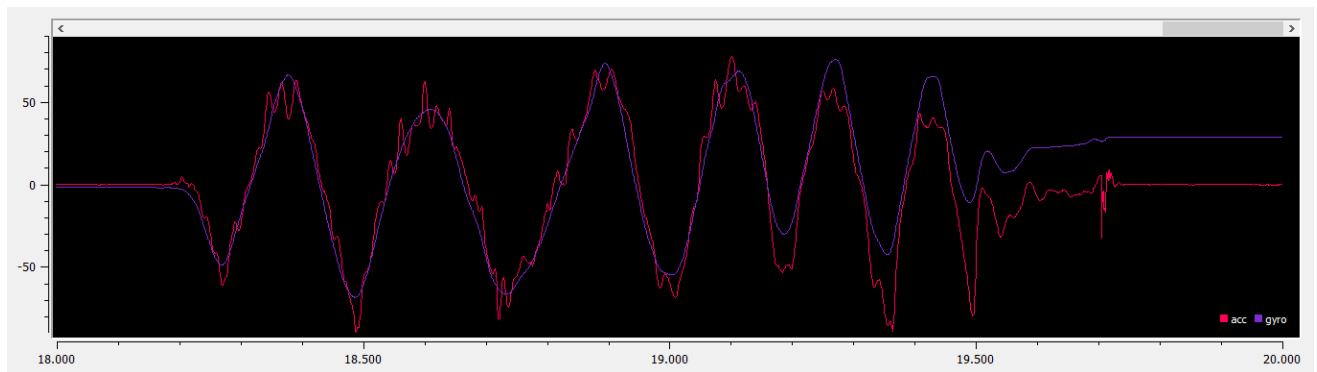
Estos métodos tienen sus ventajas y desventajas. El primero tiene mucho ruido, pero se puede confiar en sus mediciones largo plazo, el segundo tiene poco ruido, pero se desfaza por la acumulación de error al integrar, por lo que solo se puede confiar al corto plazo en sus valores.

El algoritmo que utilizamos se llama filtro de Kalman, y emplea tanto al giróscopo como al acelerómetro, y al estado anterior para hacer su estimación.

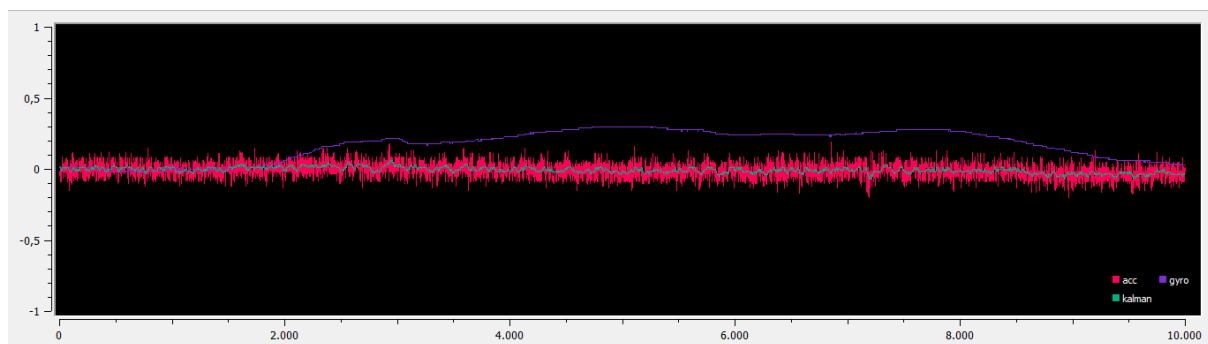
La librería que nos permite implementar el filtro de Kalman la sacamos de este [repositorio de github](#).

Luego hicimos varias pruebas comparando los distintos métodos. Los gráficos se hicieron con el programa SerialPlot.

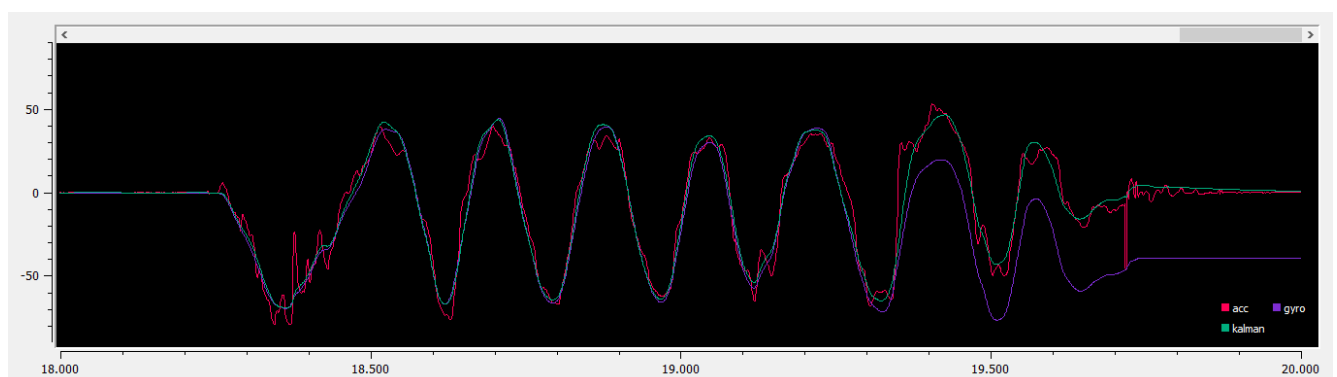
En la primera prueba compramos el método trigonométrico con el integrativo, girando el sensor de un lado a otro. Puede verse como el trigonométrico tiene grandes picos de ruido y el integrativo se termina desfasando



En la segunda prueba comparamos las tres mediciones con el sensor en reposo.

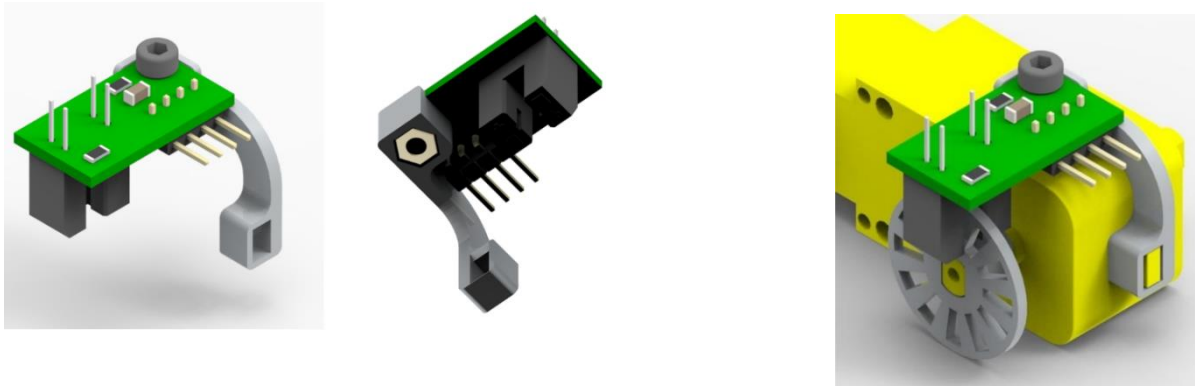


Finalmente comparamos las tres mediciones:



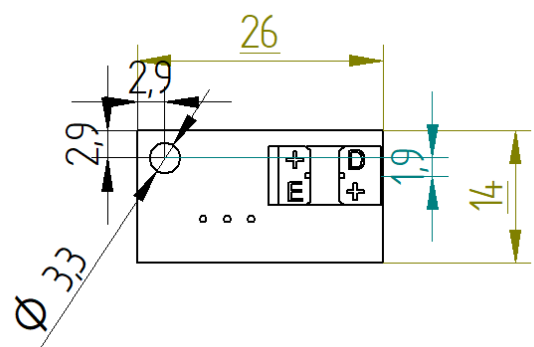
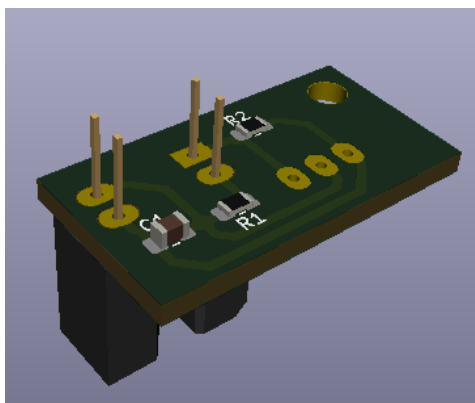
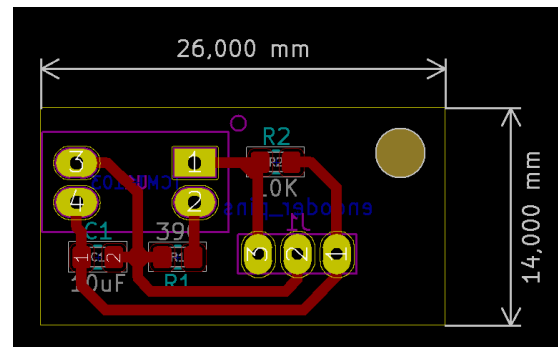
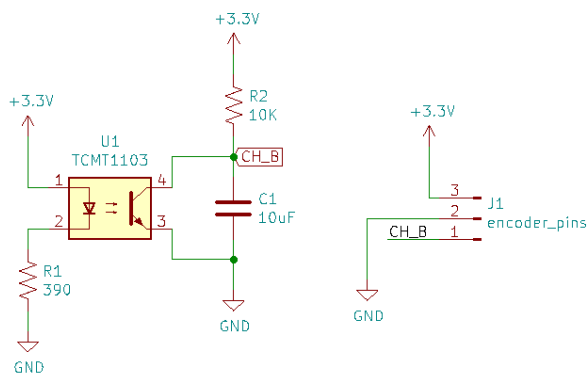
2.4. Diseño y fabricación del encoder óptico

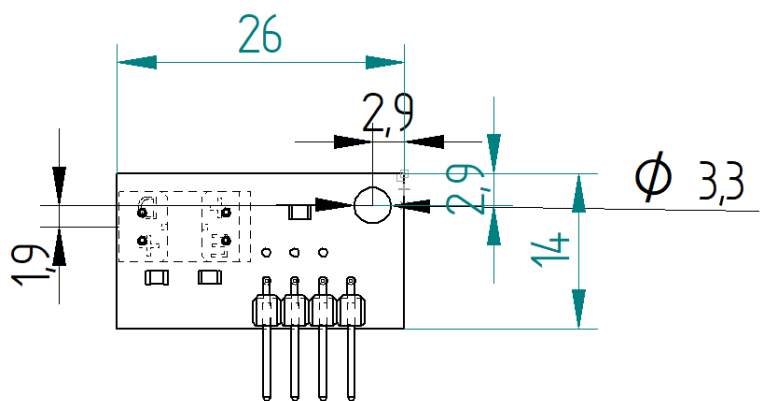
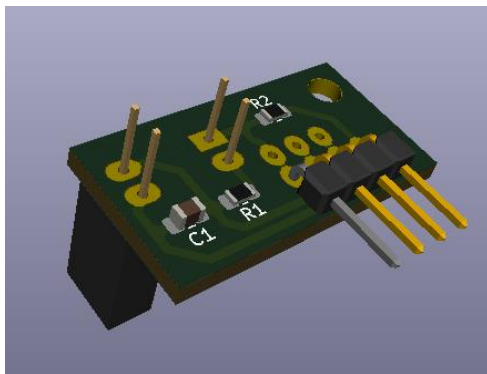
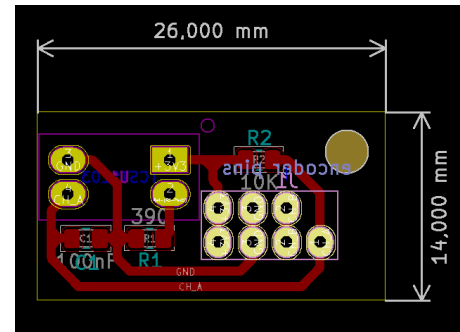
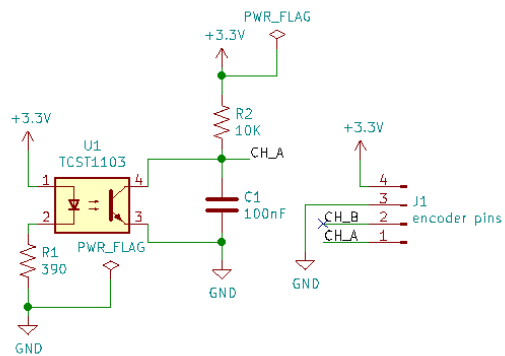
Como dijimos antes, para diseñar los encoders nos basamos en otro diseño:



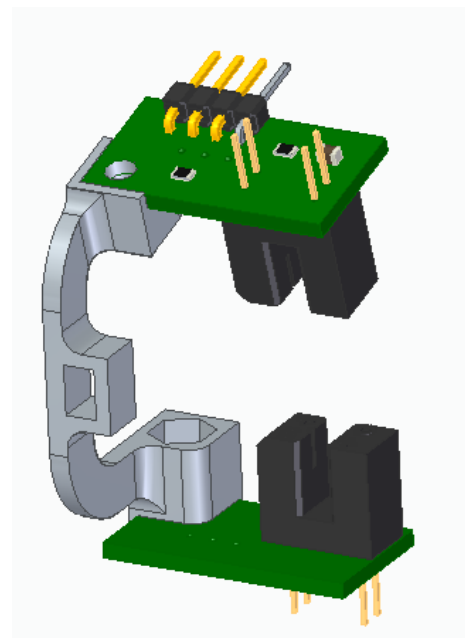
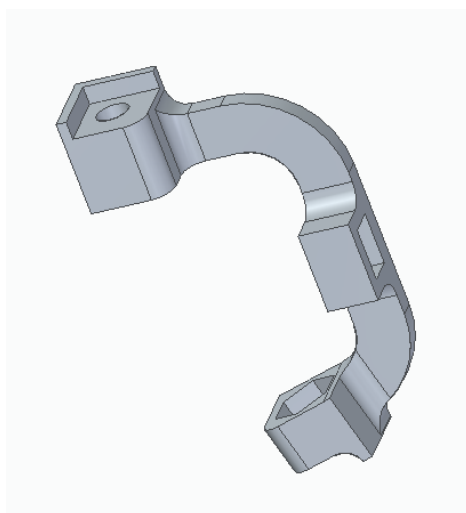
Como podemos ver, consiste de una rueda ranurada y un sensor de ranura (TCST1103). Nosotros usamos el mismo sensor.

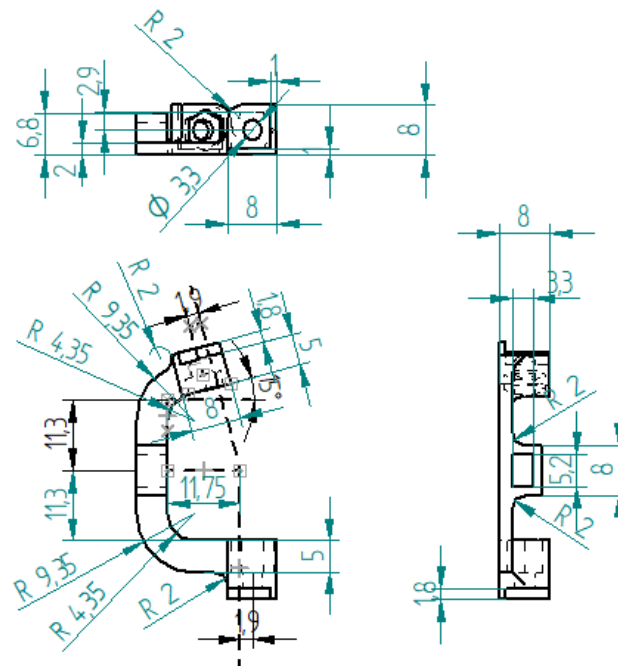
Primero diseñamos las placas:



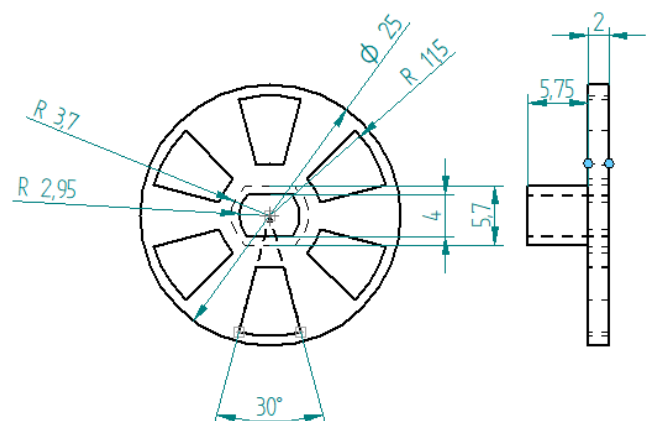
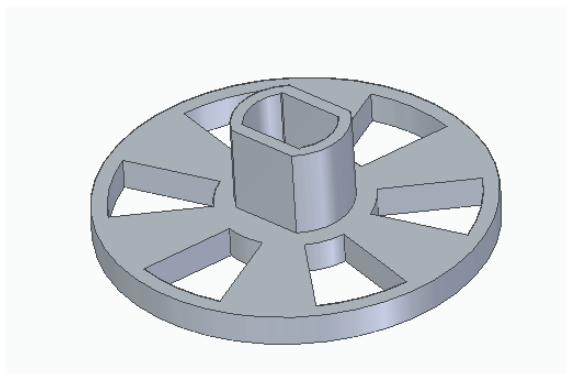


Y teniendo en cuenta sus medidas diseñamos el soporte:

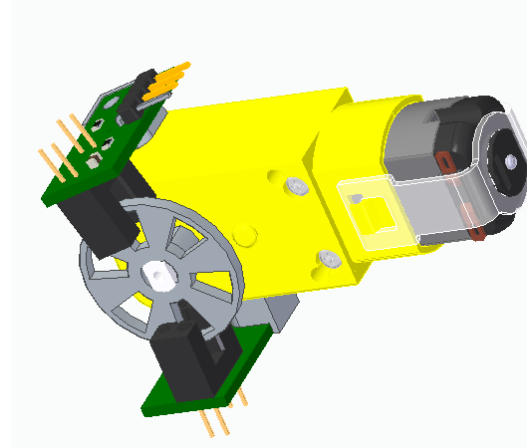
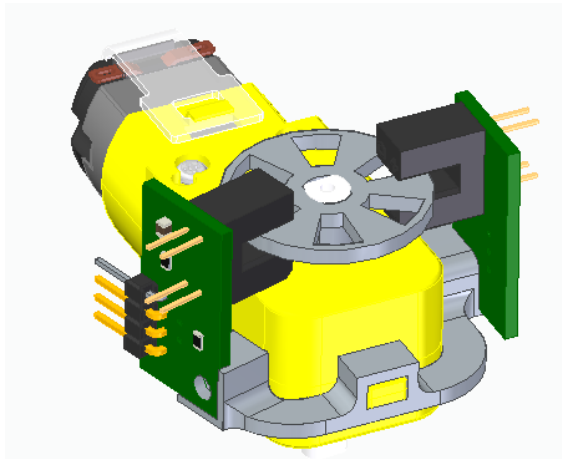




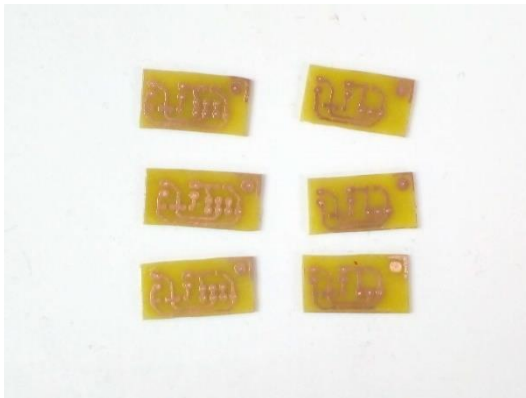
Como podemos ver, los sensores de ranura se hallan desfasados 15° , por lo que nuestra rueda ranurada es de 30° , dándonos una resolución de 24 pulsos por giro.



Se puede ver el ensamble



Luego procesamos las placas e imprimimos los soportes y las ruedas en la impresora 3D:



2.6. El código

Para el código, usamos dos loops, uno corriendo en un núcleo distinto. Un bucle se encarga de la comunicación y otro de la lógica de control, o sea, de la lectura de sensores, de los cálculos del PID, de accionar los motores.

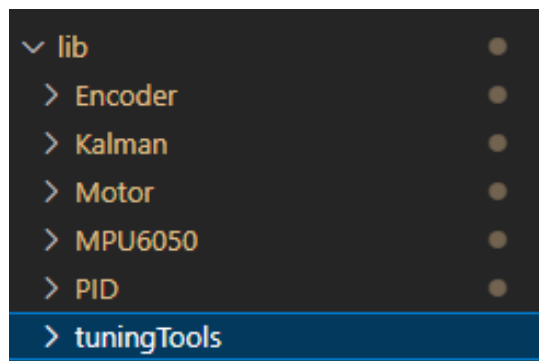
Además, separamos todo en clases. Por ejemplo, si queremos controlar al MPU6050, creamos un miembro de la clase MPU6050 (imu en nuestro código) y simplemente con llamar al método Update(dt), se actualiza el ángulo. Esto porque la comunicación I2C y los cálculos del filtro de Kalman se hacen dentro de esta función. También creamos la clase Motor y la clase Encoder. Esto nos permite modificar el código de una manera mucho más sencilla

```
void loop()
{
  #pragma region communication
  }...
  #pragma endregion
}

void loop2(void* pvParameters)
{
  const uint32_t deltaTime = 2000; //
  const float dt = deltaTime/1000000.f;
  uint32_t timer;
  int sgn = 1;
  while(1)
  {
    timer = micros();

    #pragma region logic
    imu.Update(dt);
    enc1.Update();
    pid.Update(imu.angle[0] - kenc*enc1.counter, imu.rate[0], dt);
    m1.SetSpeed(pid.out*enable);
    m2.SetSpeed(pid.out*enable);

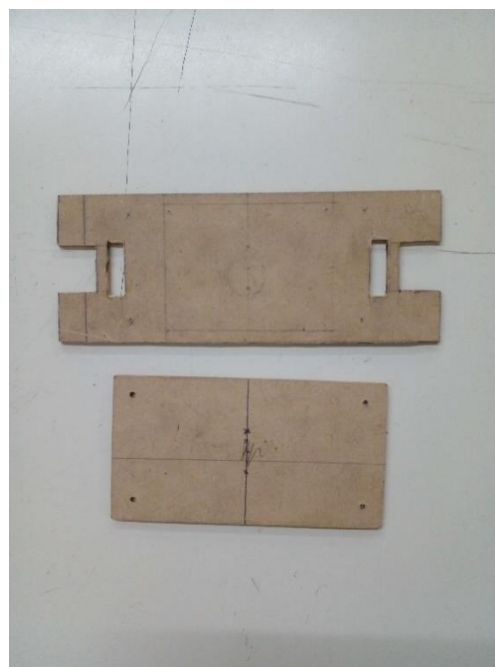
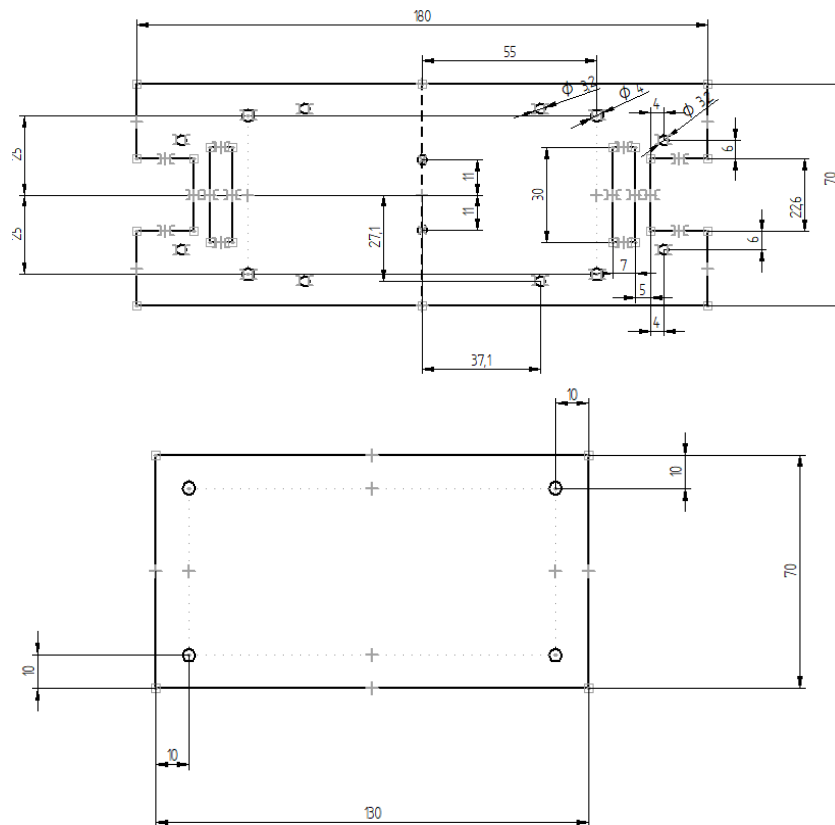
    #pragma endregion
    while (micros() - timer < deltaTime);
  }
}
```



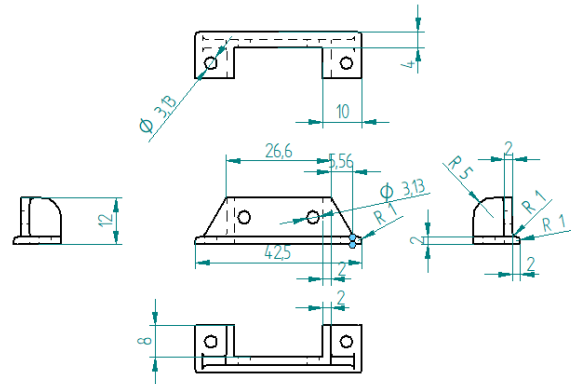
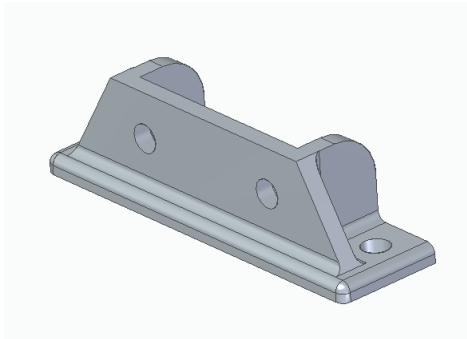
Entonces, el bucle de control corre siempre igual, cambiando en función de las variables que modifiquemos en el bucle de comunicación_AW30

2.6. Diseño y armado del chasis

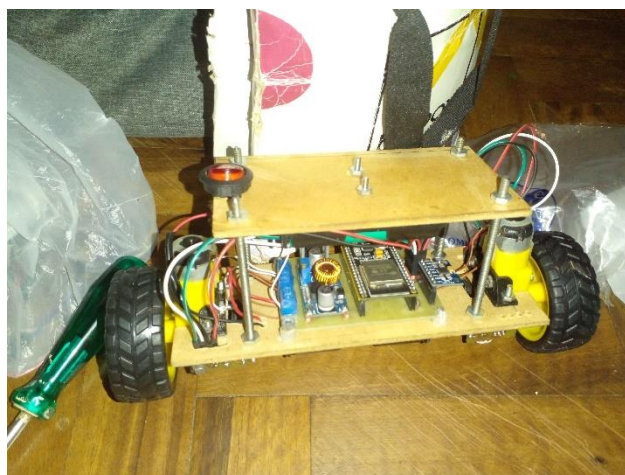
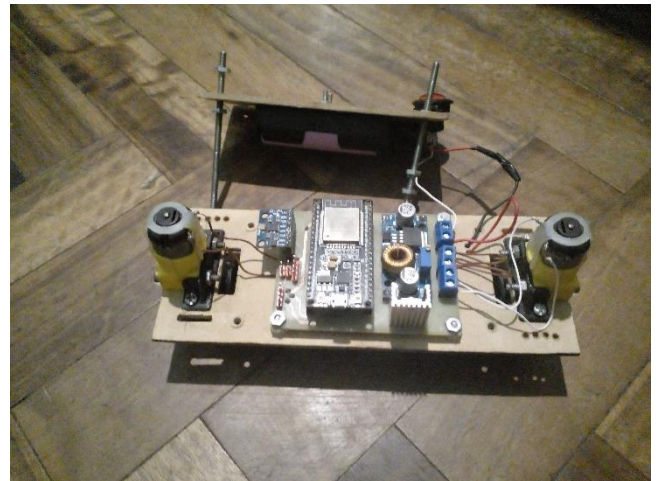
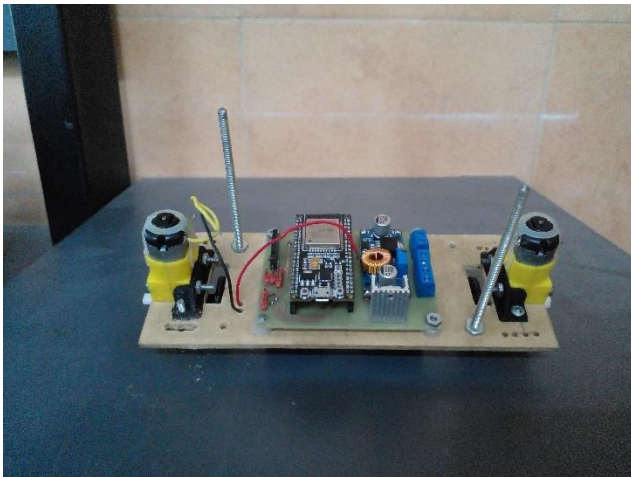
Teniendo en cuenta los encoders, diseñamos el chásis y cortamos la madera (front y back):



Luego imprimimos unos soportes para motores



Y finalmente armamos el robot:



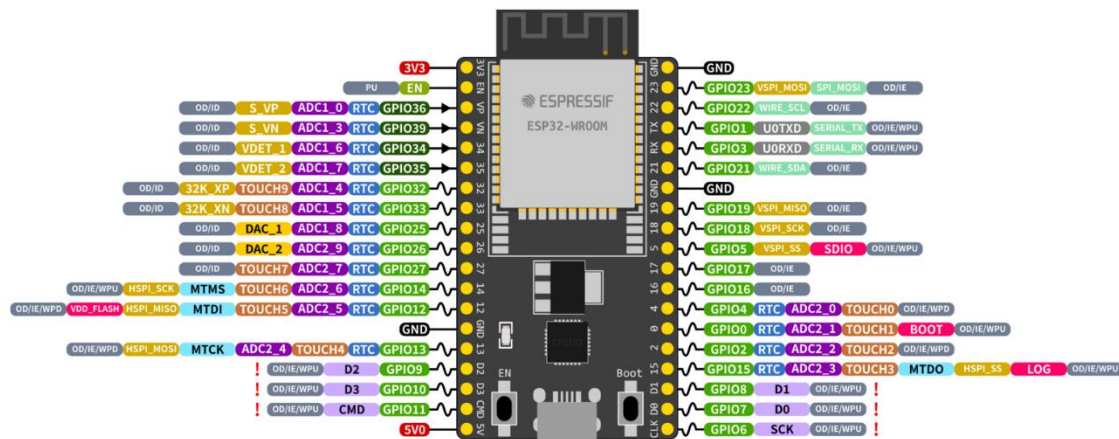
2.7. Puesta en marcha

Finalmente, cargamos el código y empezamos a calibrar el robot. Primero calibramos K_p , hasta que en un valor de 8 se mantenía oscilando con un poco de ayuda humana. Luego aumentamos K_d , hasta 0,5. En este punto la señal se vuelve suave más suave, pero si el robot se va hacia un lado no puede recomponerse. Finalmente calibramos K_i a 100. En este punto el robot se mantenía de forma estable, pero su posición no era exacta. Para eso estaban los encoders. Finalmente calibramos la constante K_{enc} a 0.1, aplicando $K_{enc} * \text{contador_encoder}$ al setpoint del PID. En este punto se alcanzó un equilibrio estable.

4. Hojas de dato e información de los componentes

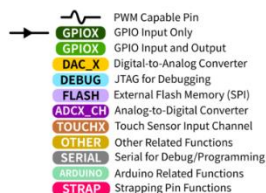
ESP32:

ESP32-DevKitC



ESP32 Specs

32-bit Xtensa® dual-core @240MHz
Wi-Fi IEEE 802.11 b/g/n 2.4GHz
Bluetooth 4.2 BR/EDR and BLE
520 KB SRAM (16 KB for cache)
448 KB ROM
34 GPIOs, 4x SPI, 3x UART, 2x I2C,
2x I2S, RMT, LED PWM, 1 host SD/eMMC/SDIO,
1 slave SDIO/SPI, TWAI®, 12-bit ADC, Ethernet



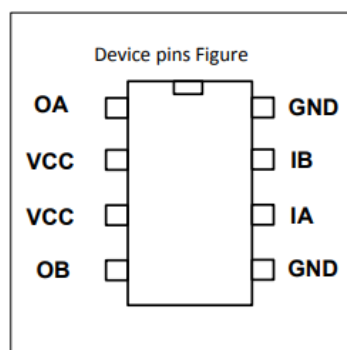
GPIO STATE
WPU: Weak Pull-up (Internal)
WPD: Weak Pull-down (Internal)
PU: Pull-up (External)
IE: Input Enable (After Reset)
ID: Input Disabled (After Reset)
OE: Output Enable (After Reset)
OD: Output Disabled (After Reset)

L9110S:

Pin definitions:

No.	Symbol	Function
1	OA	A road output pin
2	VCC	Supply Voltage
3	VCC	Supply Voltage
4	OB	B output pin
5	GND	Ground
6	IA	A road input pin
7	IB	B input pin
8	GND	Ground

Device pins Figure



Electrical characteristics:

Symbol	Parameters	Range			Units
		Min	Typical	Max	
VCC	Supply Voltage	2.5	6	12	V
Idd	Quiescent Current	—	0	2	uA
Iin	Operating current	200	350	500	uA
IC	Continuous	750	800	850	mA
IMax	Current peak	—	1500	2000	mA

Logical relationship:

IA	IB	OA	OB
H	L	H	L
L	H	L	H
L	L	L	L
H	H	L	L

XL4005 (step-down)

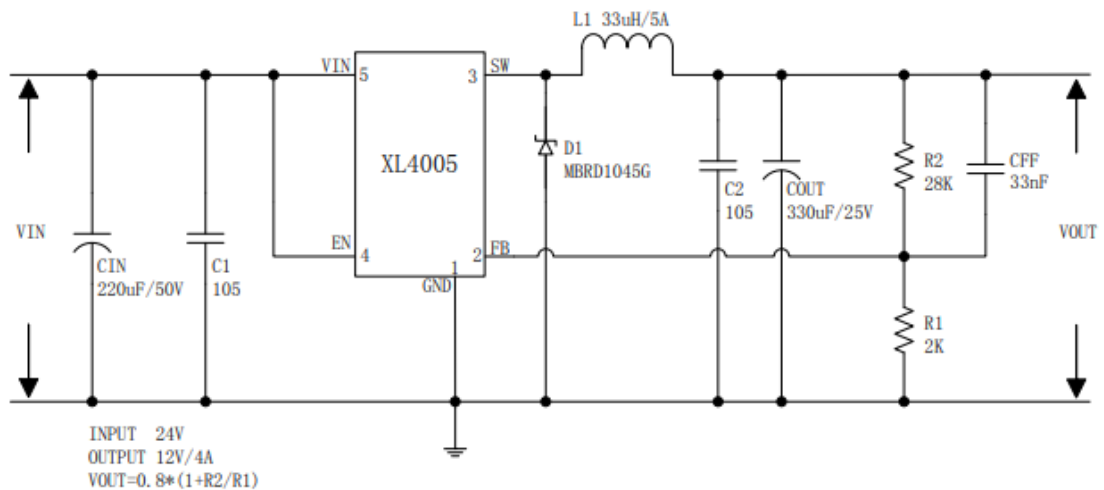
Electrical Characteristics (DC Parameters)

$V_{in} = 12V$, $GND = 0V$, V_{in} & GND parallel connect a 220uF/50V capacitor; $I_{out} = 500mA$, $T_a = 25^\circ C$; the others floating unless otherwise specified.

Parameters	Symbol	Test Condition	Min.	Typ.	Max.	Unit
Input operation voltage	V_{in}		5		32	V
Shutdown Supply Current	I_{STBY}	$V_{EN} = 0V$		60	200	μA
Quiescent Supply Current	I_q	$V_{EN} = 2V$, $V_{FB} = V_{in}$		3	5	mA
Oscillator Frequency	F_{osc}		240	300	360	Khz
Switch Current Limit	I_L	$V_{FB} = 0$		8		A
EN Pin Threshold	V_{EN}	High (Regulator ON) Low (Regulator OFF)		1.4 0.8		V
EN Pin Input Leakage Current	I_H	$V_{EN} = 2V$ (ON)		1	15	μA
	I_L	$V_{EN} = 0V$ (OFF)		1	15	μA
Max. Duty Cycle	D_{MAX}	$V_{FB} = 0V$		100		%

El circuito de nuestro módulo:

Typical System Application for 24V ~ 12V/4A Version



6.6 LM340 / LM7805 Electrical Characteristics, $V_O = 5\text{ V}$, $V_I = 10\text{ V}$

$0^\circ\text{C} \leq T_J \leq 125^\circ\text{C}$ unless otherwise specified⁽¹⁾

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
V_O	Output voltage	$T_J = 25^\circ\text{C}$, $5\text{ mA} \leq I_O \leq 1\text{ A}$	4.8	5	5.2	V
		$P_D \leq 15\text{ W}$, $5\text{ mA} \leq I_O \leq 1\text{ A}$	4.75		5.25	V
		$7.5\text{ V} \leq V_{IN} \leq 20\text{ V}$				
ΔV_O	Line regulation	$I_O = 500\text{ mA}$		3	50	mV
		$T_J = 25^\circ\text{C}$ $7\text{ V} \leq V_{IN} \leq 25\text{ V}$				
		Over temperature			50	mV
		$8\text{ V} \leq V_{IN} \leq 20\text{ V}$				
ΔV_O	Load regulation	$I_O \leq 1\text{ A}$			50	mV
		$T_J = 25^\circ\text{C}$ $7.5\text{ V} \leq V_{IN} \leq 20\text{ V}$				
		Over temperature			25	mV
		$8\text{ V} \leq V_{IN} \leq 12\text{ V}$				
ΔV_O	Load regulation	$T_J = 25^\circ\text{C}$		10	50	mV
		$5\text{ mA} \leq I_O \leq 1.5\text{ A}$				
		$250\text{ mA} \leq I_O \leq 750\text{ mA}$			25	mV
I_Q	Quiescent current	Over temperature, $5\text{ mA} \leq I_O \leq 1\text{ A}$			50	mV
		$I_O \leq 1\text{ A}$				
		$T_J = 25^\circ\text{C}$			8	mA
ΔI_Q	Quiescent current change	Over temperature			8.5	mA
		$0^\circ\text{C} \leq T_J \leq 125^\circ\text{C}$, $5\text{ mA} \leq I_O \leq 1\text{ A}$		0.5		mA
		$7\text{ V} \leq V_{IN} \leq 20\text{ V}$			1	mA
V_{IN}	Output noise voltage	$T_J = 25^\circ\text{C}$, $I_O \leq 1\text{ A}$			1	mA
		Over temperature, $I_O \leq 500\text{ mA}$			1	mA
		$7\text{ V} \leq V_{IN} \leq 20\text{ V}$				
$\frac{\Delta V_{IN}}{\Delta V_{OUT}}$	Ripple rejection	$T_A = 25^\circ\text{C}$, $10\text{ Hz} \leq f \leq 100\text{ kHz}$		40		μV
		$f = 120\text{ Hz}$		62	80	dB
		$8\text{ V} \leq V_{IN} \leq 18\text{ V}$		62		dB
R_O	Dropout voltage	$T_J = 25^\circ\text{C}$, $I_O = 1\text{ A}$		2		V
	Output resistance	$f = 1\text{ kHz}$		8		m Ω
	Short-circuit current	$T_J = 25^\circ\text{C}$		2.1		A
	Peak output current	$T_J = 25^\circ\text{C}$		2.4		A
	Average TC of V_{OUT}	Over temperature, $I_O = 5\text{ mA}$		-0.6		mV/ $^\circ\text{C}$
V_{IN}	Input voltage required to maintain line regulation	$T_J = 25^\circ\text{C}$, $I_O \leq 1\text{ A}$	7.5			V

TSCT1103

ABSOLUTE MAXIMUM RATINGS ($T_{amb} = 25^\circ\text{C}$, unless otherwise specified)				
PARAMETER	TEST CONDITION	SYMBOL	VALUE	UNIT
INPUT (EMITTER)				
Reverse voltage		V_R	6	V
Forward current		I_F	60	mA
Forward surge current	$t_p \leq 10\text{ }\mu\text{s}$	I_{FSM}	3	A
Power dissipation	$T_{amb} \leq 25^\circ\text{C}$	P_V	100	mW
Junction temperature		T_J	100	$^\circ\text{C}$
OUTPUT (DETECTOR)				
Collector emitter voltage		V_{CEO}	70	V
Emitter collector voltage		V_{ECO}	7	V
Collector peak current	$t_p/T = 0.5$, $t_p \leq 10\text{ ms}$	I_{CM}	200	mA
Power dissipation	$T_{amb} \leq 25^\circ\text{C}$	P_V	150	mW
Junction temperature		T_J	100	$^\circ\text{C}$

5. Conclusiones

Realizando esta práctica aprendimos como implementar un sistema de control utilizando varios sensores.

Fue interesante observar la comparación entre los distintos métodos de medir el ángulo, y ver como el filtro de Kalman superaba las desventajas de los demás métodos.

Finalmente, pudimos ver intuitivamente qué hacían las constantes PID sobre el robot.

La constante K_p permitía que oscile, pero no le daba un equilibrio estable, la K_d suavizaba más el movimiento (al oponerse a los cambios bruscos) y la K_i daba el ajuste fino; Finalmente, los encoders permitían mantener la posición exacta.

6. Referencias

- https://es.wikipedia.org/wiki/Controlador_PID
- <https://robotics.stackexchange.com/questions/17964/self-balancing-robot-pid-control-algorithm>
- <https://arxiv.org/ftp/arxiv/papers/2303/2303.12067.pdf>
- <https://lastminuteengineers.com/esp32-pwm-tutorial/#:~:text=PWM%20Channels,-Now%20let%20us&text=A%20channel%20represents%20a%20unique,and%208%20low-speed%20channels.>
- <https://randomnerdtutorials.com/esp32-pwm-arduino-ide/>
- https://es.wikipedia.org/wiki/Unidad_de_medición_inercial
- https://www.youtube.com/watch?v=7VW_XVbtu9k
- <https://www.youtube.com/watch?v=5HuN9iL-zxU>
- <http://blog.tkjelectronics.dk/2012/09/a-practical-approach-to-kalman-filter-and-how-to-implement-it/>